Parcial Algoritmos y Programación II 7541 Curso Calvo (1º instancia) 3 de Junio de 2013

Padrón: **Apellido y Nombre:**

Punteros: APROBADO - DESAPROBADO

1) Indicar la salida por pantalla y escribir las sentencias necesarias para liberar correctamente la memoria.

```
int main(){

  int *A, *C, *F;
  int **B;
  int H = 20;

  A = new int[3];

  for (int i = 0; i < 3; i++) {
      A[i] = H + i;
  }

  C = new int;
  (*C) = A[1];
  F = A + 2;

  cout << (*F) << (*C) << A[0] << endl;

  B = new int*[3];
  B[0] = C;
  B[1] = F;
  B[2] = &H;

  cout << *B[1] << **B << *B[2] << endl;

  (*B[0]) = (*F) + 3;
  H++;
  A[2] = (*C) + 1;

  cout << *C << *B[1] <<  *B[2] << endl;

  F = C;
  C = A + 2;
  (**B) = A[0];
  A[2] = ** (B + 2);

  cout << H << (*C) << (*F) << endl;

  // liberar la memoria
  // ...

  return 0;
}
```

2. Desarrollar una implementación dinámica del TDA Cola.

3. Implementar el método seleccionarVideo de la clase Editor a partir de las siguientes especificaciones:

```
class Editor {

  public:

    /* post: devuelve una nueva lista con aquellos videos que tengan la etiqueta indicada y su promedio de calificaciones sea
     *       superior a calificacionPromedioMinima.
     */
    Lista<Video*>* seleccionarVideos(Lista<Video*>* videos, int calificacioPromedioMinima, String etiqueta);

};
```

```
class Video {

  public:
    /* post: inicializa el Video con el título y URL.
     */
    Video(string titulo, string url);
    /* post: devuelve el título del Video.
     */
    string getTitulo();
    /* post: devuelve la URL del Video.
     */
    string getUrl();
    /* post: devuelve los comentarios asociadas al Video.
     */
    Lista<Comentario*>* getComentarios();
    /* post: devuelve las etiquetas del video
     /
    List<String>* getEtiquetas();
    ~Video();
};
```

```
class Comentario {

  public:
    /* post: inicializa el Comentario con el contenido
     *       y calificación 0.
     */
    Comentario(string contenido);

    string getContenido();
    /* post: devuelve la calificación [1 a 10] asociada,
     *       o 0 si el Comentario no tiene calificación.
     */
    int getCalificacion();
    /* pre : calificacion está comprendido entre 1 y 10
     * post: cambia la calificación del Comentario.
     */
    void calificar(int calificacion);
};
```

4. Diseñar la especificación e implementar el TDA **Cerrojo**. El Cerrojo posee tres estados posibles: ABIERTO, CERRADO o BLOQUEADO. Un Cerrojo debe crearse con un código de seguridad secreto y debe bloquearse luego de haber realizado más de 10 intentos fallidos consecutivos por abrirlo. Una vez que el Cerrojo está BLOQUEADO no es posible abrirlo nunca más. Debe proveer operaciones para:

- conocer su estado
- intentar abrirlo proporcionando un código
- cerrarlo
- devolver la cantidad de aperturas exitosas realizadas
- devolver la cantidad de intentos restantes antes de pasar al estado BLOQUEADO
- cambiar el código de seguridad (solo en caso de estar ABIERTO)
- devolver la cantidad máxima de intentos fallidos consecutivos en toda la vida de Cerrojo.

Los alumnos que tienen aprobado el parcialito de punteros no deben realizar el ejercicio 1.
Para aprobar es necesario tener al menos el 60% de los ejercicios correctos y completos.
Para cada método escribir pre y post condición, si recibe argumentos y cuáles, y si retorna un dato y cuál. De faltar ésto, se considerará el código incompleto.
Duración del examen : 3 horas